

Read an HTTP request

Read the text message every web request really is, and pull out the useful parts.

The big idea

When you open a website, your browser sends a short **text message** to a server: an **HTTP request**. One page needs many of them: one for the HTML, then more for each image, CSS and JavaScript file.

A backend's first job is to **read** that message. It's like cleaning data: one messy string in, tidy labelled fields out.

What a request looks like

This is the exact text the tests pass to your function as `raw`. `\r\n` means "new line".

```
raw = (  
    "GET /api/tasks/?status=done&page=2 HTTP/1.1\r\n"      # request line  
    "Host: localhost:8000\r\n"                             # headers...  
    "Accept: application/json \r\n"  
    "\r\n"                                                  # blank line = end of head  
    )                                                       # body: empty for GET
```

New words

method

The verb: `GET` reads something, `POST` creates something.

path

The address that was asked for, like `/api/tasks/`.

query string

Extras after `?`, like `status=done&page=2`.

headers

Name: `value` lines with extra information.

head / body

Everything before / after the first blank line.

parameter

The name a function gives its input, like `raw`.

What your function must return

`parse_request(raw)` turns the text into one dictionary:

```
{  
  "method": "GET",  
  "path": "/api/tasks/",  
  "query": {"status": "done", "page": "2"},  
  "version": "HTTP/1.1",  
  "headers": {"host": "localhost:8000", "accept": "application/json"},  
  "body": "",  
}
```

Tool: partition splits once

`text.partition(sep)` is a **method**: it cuts at the **first** separator and gives back 3 pieces.

```
before, sep, after = "name: Ada Lovelace".partition(": ")
print(before)
print(after)

name, _, value = "Host: localhost:8000".partition(":")
print(value.strip())
```

OUTPUT

```
name
Ada Lovelace
localhost:8000
```

split vs partition

text.split(sep)

- Cuts at **every** separator
- Gives back a **list**
- `"a,b,c".split(",")` gives `["a", "b", "c"]`
- Good for: splitting the head into lines

text.partition(sep)

- Cuts at the **first** separator only
- Always gives back **3 pieces**
- `"a:b:c".partition(":")` gives `("a", ":", "b:c")`
- Good for: head vs body, and each header line

Tools: urlsplit and parse_qs

Both are **functions**: pass the text in brackets. `urlsplit` results use dots (`.path`), not `["path"]`.

```
from urllib.parse import parse_qs, urlsplit

parts = urlsplit("/shop/search/?item=tea&size=large")
print(parts.path)
print(parts.query)
print(dict(parse_qs(parts.query)))
```

OUTPUT

```
/shop/search/
item=tea&size=large
{'item': 'tea', 'size': 'large'}
```

Pattern: loop and build a dictionary

The same shape you need for the headers, on different data:

```
lines = ["Fruit : Apple ", "Colour: Red"]
info = {}
for line in lines:
    key, _, value = line.partition(":")
    info[key.strip().lower()] = value.strip()
print(info)
```

OUTPUT

```
{'fruit': 'Apple', 'colour': 'Red'}
```

Your task, step by step

- 1 Split `raw` into head and body at `"\r\n\r\n"` with `partition` → **test 7**
- 2 Split the head into lines; the first one is the request line
- 3 Split the request line into method, target and version → **test 1**
- 4 `urlsplit(target)` gives the path; `dict(parse_qs1(...))` gives the query → **tests 2-4**
- 5 Loop over header lines and build `{lower-case name: stripped value}` → **tests 5-6**
- 6 `is_safe_method`: upper-case the method, then check it's in `{"GET", "HEAD", "OPTIONS"}` → **tests 8-9**

Common mistakes

- `raw.urlsplit(...)`: `urlsplit` is a function, so write `urlsplit(target)`
- `parts["path"]`: `urlsplit` results use dots, `parts.path`
- `.split(":")` on `Host: localhost:8000` gives 3 pieces; use `partition`
- Forgetting `dict(...)` gives a list of pairs, not a dictionary
- `NameError: raw is not defined` in the Console: make your own sample string first

Where you'll use this

A to-do app asks your server: `GET /api/tasks/?status=done`. Before your code can answer, something has to work out that the client wants to **read** tasks, only the **done** ones.

- Django does this parsing for you: `request.method`, `request.path`, `request.GET`, `request.headers`, `request.body`
- `is_safe_method` is how servers decide what anonymous visitors may do: reading is often allowed, changing is not
- Knowing the raw format helps you debug APIs with browser dev tools or `curl`

Check yourself

1. What separates the head of a request from its body?
2. Why use `partition(":",)` instead of `split(":",)` for header lines?
3. Is `urlsplit` a function or a method? How do you read the path from its result?
4. Why do `GET` and `HEAD` count as safe, but `POST` doesn't?

Keep learning

[What is HTTP? Protocol Overview for Beginners](#)

freeCodeCamp · article · beginner. Start here: a friendly walk through what a request looks like, with the method, path, version, headers and body explained one by one.

[Overview of HTTP](#)

MDN Web Docs · article · beginner. The big picture of how a client and a server talk, so you know where the raw request you parse comes from.

[HTTP messages](#)

MDN Web Docs · article · beginner. Shows the exact layout of a request line, headers, the blank line and the body: the same structure your parser splits apart.

[HTTP request methods](#)

MDN Web Docs · docs · beginner. A reference list of GET, POST, PUT, PATCH, DELETE and friends, including which ones are safe, for your `is_safe_method` function.

[urllib.parse: Parse URLs into components](#)

Python docs · docs · intermediate. The official reference for `urlsplit` and `parse_qs`, the two functions you use to separate the path from the query string.